FULL-STACK WEB APPLICATION | PROJECT REPORT

LinkLite

A secure, SQL Server-backed URL shortener

SHORTEN	MANAGE	MEASURE

Project overview, system design, implementation, security, testing and deployment considerations.

Prepared from the LinkLite source code and project documentation

November 2026

Executive Summary

LinkLite is a full-stack web application for creating and managing short URLs. The project combines a responsive browser interface with a Node.js and Express backend, Microsoft SQL Server persistence, user authentication, click tracking, and administrative tools. Visitors can use guest mode; registered users get a private workspace for their links.

The implementation generates short codes, validates destination URLs, stores links and ownership data, redirects visitors to original destinations, and records click metrics. Authentication uses bcrypt password hashing and server-side SQL-backed sessions. The repository also contains a static GitHub Pages preview; that preview is not connected to the backend and cannot create accounts or shorten links.

1. Introduction

Long URLs can be awkward to share and difficult to manage. LinkLite addresses this by providing a web-based workflow for shortening a destination, organizing created links, and viewing basic engagement metrics. The project is designed as a practical, self-hostable application rather than a static mockup.

2. Problem Statement

Users need a simple way to transform long web addresses into concise links while retaining control over their link history and access. A usable solution must also persist records, handle redirects reliably, distinguish link ownership, and provide basic account and administration capabilities.

3. Aim and Objectives

- Build a responsive interface for authentication and link management.
- Create short links with generated codes or optional custom aliases.
- Persist destination, ownership, creation time and click statistics in SQL Server.
- Provide private user workspaces and role-gated administration.
- Apply input validation, password hashing, session controls and request limits.

4. Scope

Included in this implementation	Not provided by the project alone
URL creation, custom endings, redirect and click counts	Guaranteed uptime or a permanent public hosting service
Guest and registered-user link ownership	Email verification, password reset or identity proofing
Admin workspace for user/link oversight	Automated abuse detection or a destination blocklist
Local SQL Server and deployment guidance	Third-party availability, backups or a service-level agreement

5. System Overview and Features

The browser sends API requests to the Express application. The backend validates input, applies authentication and ownership rules, and reads or writes records in SQL Server. A short-code request looks up the original destination and responds with an HTTP 302 redirect.

Browser UI HTML, CSS, JavaScript	**Express API** Validation, auth, routing	**SQL Server** Users, links, sessions
Create, search, copy, sign in	Persist data and issue redirect	Store records and click metrics

Core user capabilities

Capability	How it works
Shorten a URL	Accepts HTTP/HTTPS destinations; generates a seven-character code or accepts an optional custom alias.
Manage links	Lists a user or guest?s own links, supports search, and allows deletion by the owner.
Redirect and metrics	A short-code route issues a non-cacheable 302 redirect and increments the click count.
Accounts	Registration, sign-in, sign-out, session inspection and profile/account operations.
Administration	Role-protected overview, account search/disable and link search/removal.

Representative API surface

Method and path	Purpose
POST /api/auth/register	Create a standard user account
POST /api/auth/login; POST /api/auth/logout	Start or end an authenticated session
POST /api/urls; GET /api/urls	Create and list owned links
GET /api/urls/summary; DELETE /api/urls/:id	Read link totals and delete an owned link
GET /:code; GET /health	Redirect and provide database-backed health status

Public registration creates standard users; administrator provisioning is intentionally handled through private configuration or a trusted database operation.

6. Data Model and Information Flow

The SQL Server schema separates link records, user accounts and server-side sessions. A unique index protects short-code uniqueness; ownership and creation-time indexes support user link listing.

Table	Key fields and purpose
dbo.urls	id (BIGINT), owner_id, short_code, original_url, clicks, created_at, last_clicked_at. Stores destinations, ownership and usage counters.
dbo.users	id, display_name, email, password_hash, role, is_active, session_version, created_at. Stores accounts and authorization state.
dbo.auth_sessions	sid, session_data, expires_at. Persists server-side login sessions and expiration.

7. Data Dictionary and Rules

Concept	Rule / handling
Short code	Seven random alphanumeric characters by default; custom aliases are 3-32 letters, digits, underscore or hyphen. Comparison is case-sensitive.
Destination URL	Only HTTP and HTTPS are accepted; input is normalized and limited to 2,048 characters. Self-referential links are rejected.
Ownership	Authenticated links belong to a user ID; guest links are scoped to a browser-generated client ID.
Click metric	Integer counter incremented on redirect; last-clicked time is stored in UTC.
Credentials	Passwords are validated, hashed with bcrypt and never stored as plaintext.
Session	Session data is held in SQL Server with a seven-day default lifetime and expiry cleanup.

Processing flow

- The browser submits a destination and optional alias with the user session or guest client ID.
- The API validates URL and alias syntax, checks reserved names, and inserts a unique code.
- The owner?s workspace loads only that owner?s links; deletion is scoped by both record and owner.
- A visitor follows a code; the server resolves the destination, updates metrics and responds with a redirect.

No production dataset or usage totals are asserted in this report. Metrics shown by the application depend on the database records present at runtime.

8. Security and Privacy

The implementation includes several baseline protections appropriate to an account-enabled link service. These measures reduce common risks but do not replace a deployment review, operational monitoring or abuse controls.

Control	Implementation purpose
Password hashing	bcryptjs stores salted password hashes rather than plaintext passwords.
Session security	SQL-backed server-side sessions and HTTP-only, same-site cookies support authenticated requests.
Authorization	User routes are scoped to the signed-in user or guest client; admin routes require the admin role.
Parameterized SQL	Values are bound as SQL parameters to avoid building queries from untrusted input.
HTTP headers	Helmet adds security-related response headers; production mode trusts the configured hosting proxy.
Rate limiting	Registration, authentication and link creation endpoints apply request limits.
Input validation	Validates URLs, aliases, identifiers, email, password length and JSON request bodies.
Loop prevention	Rejects a destination that points back to the LinkLite host.

Operational privacy guidance

- Keep .env and database credentials private; never commit them or paste them into support chats.
- Use HTTPS and a strong, unique session secret in hosted environments.
- Use a least-privilege SQL login for the app; do not expose SQL Server directly to the public internet.
- Treat links and click data as potentially sensitive; restrict database access and arrange backups.
- A public shortener can be abused. Consider abuse reporting, destination screening, account verification and retention policies before opening the service broadly.

The project does not claim to implement email verification, password reset, a malware/phishing blocklist, or a full audit trail. These should be considered before production use.

9. Testing and Verification Status

During the local support session, the SQL connection and /health endpoint were verified successfully. A temporary HTTPS tunnel was also confirmed to reach the running application. These are point-in-time operational checks, not a substitute for automated unit, integration, load or security testing. No broader test-suite result is claimed here.

10. Deployment and Operation

The full application requires a Node.js runtime and a reachable SQL Server database. The repository describes local SQL Server setup, app environment variables, schema initialization and hosted deployment options.

Mode	What it provides	Important constraint
Local development	Node.js/Express plus local SQL Server Express or Developer instance.	Reachable only on the developer?s machine unless deliberately exposed.
GitHub Pages	Static UI preview from the docs folder.	Does not run the API, authentication, shortener or database.
Quick tunnel	Temporary HTTPS route to a local app for demos.	Depends on the PC, app and tunnel remaining online; URL may change. Not production hosting.
Hosted app + database	Public app runtime and managed/reachable SQL Server.	Requires provider setup, secure configuration, monitoring and may incur costs.

Local run checklist

- Install Node.js 18 or later and project packages.
- Create the linklite database and schema; configure the limited SQL login.
- Set DB_SERVER, DB_INSTANCE or DB_PORT, DB_NAME, DB_USER and DB_PASSWORD in the local .env file.
- Set a random SESSION_SECRET of at least 32 characters; configure admin bootstrap only when required.
- Start with npm run dev and verify the app?s /health endpoint before using account or link features.

11. Limitations and Future Work

Current limitation	Suggested next step
Demo tunnel is transient and tied to a local computer.	Deploy to a stable host with HTTPS, monitoring and backups.
No built-in destination reputation screening.	Add an abuse reporting and URL screening workflow.
No email verification or password recovery flow.	Design secure account recovery and verification before public launch.
Click count is basic and may not represent unique human visits.	Document metric semantics; consider privacy-preserving analytics.
No automated test results are included in this report.	Add API and database integration tests plus deployment checks.
Static preview is disconnected from the backend.	Clearly link users to the hosted app once a permanent deployment exists.

12. Conclusion

LinkLite brings together URL shortening, redirect handling, ownership-aware link management, account sessions and administrative oversight in a compact Node.js and SQL Server application. Its design demonstrates a complete web application flow: browser interaction, API validation, persistence, authorization and tracked redirects.

The repository also publishes a static GitHub Pages preview, but that preview alone cannot provide functional account creation or URL shortening. A temporary tunnel can expose a locally running backend for demonstration, but it is not a permanent public service. Reliable sharing requires hosting the application and database with secure configuration and ongoing operations.

13. Quick User Guide

Task	Action
Open the full app	Use the active hosted/tunnel URL, or http://localhost:3000 on the same computer that runs the server.
Create an account	Select Create account, provide a name and valid email, and choose a password meeting the minimum length.
Shorten a URL	Sign in or continue as guest, paste an HTTP/HTTPS URL, optionally set an alias, then create the link.
Share and track	Copy the generated short URL; redirects update the link?s click count.
Use a temporary demo	Keep the app, SQL Server and tunnel running. Do not rely on a quick-tunnel URL after it is stopped.

14. Project Files and References

Resource	Role
server.js	Express routes, authentication, link logic, redirect, error handling and startup.
config/db.js	SQL Server configuration and shared connection pool.
database/schema.sql	Database tables and indexes.
public/	Full browser application assets.
docs/	Static GitHub Pages preview assets.
README.md and .env.example	Setup, deployment guidance and documented configuration keys.

Report basis: repository source files and README documentation supplied with the LinkLite project; runtime checks were limited to those explicitly performed during the support session. This report does not include fabricated user counts, performance benchmarks or test coverage claims.